

Tarea RA5:

Qué problemas aparecen cuando presentación y lógica de negocio están mezcladas;

Los problemas que detecto cuando lógica y presentación están juntos es que no se podrían reutilizar las funciones creadas ya que solo pertenecen a este archivo y no a un modelo; deberías copiarlo y pegarlo en todos los archivos que lo necesitaras. Si queremos hacer algún tipo de cambio en el HTML, tendríamos que a lo mejor modificar alguna estructura dentro del archivo para que aplicara bien la lógica, mientras que si estuviera separado tendríamos solo que cambiar el HTML. Al tener tanta lógica como validaciones en el mismo archivo, para comprobar una de las dos o incluso una función tendríamos que cargar el archivo entero y esto sería una forma bastante lenta en comparación con comprobar solamente lo que queremos. Por último, a la hora de encontrar un error o depurar tendríamos que fijarnos en muchas líneas al estar todo junto y si este ejercicio escalara mucho más se convertiría en un código muy extenso y sería aún peor.

Qué ventajas aporta separar controladores, modelos/servicios y vistas;

Como ventaja principal yo diría la reutilización del código en cualquier archivo de este proyecto, por lo tanto una optimización de este. Separamos el código de la siguiente forma: en el modelo añadiremos los datos que necesita el modelo (objeto) según necesidad y la lógica de negocio, vista de la página que va a ver el usuario en este ejercicio los formularios y en el controlador recogemos esta información del formulario mediante método POST y llamamos al modelo para poder utilizarlo a este y sus funciones. Luego un mayor orden y estructuración del proyecto.

Qué papel cumple cada parte de una arquitectura tipo MVC;

Modelo: almacena los datos y la lógica de negocio que utilizaremos durante el ejercicio en la clase Incidencia.

Controlador: es el que obtiene la información del formulario (vista) y, junto al modelo, valida los datos que pasamos por el formulario y llama a los métodos del modelo para utilizarlos.

Vista: es lo que va a ver el usuario por pantalla; es básicamente el HTML separado de la lógica y solicitudes a la base de datos.

Cómo mejora esto el mantenimiento, la reutilización y las pruebas.

La reutilización lo mejora ya que, simplemente instanciando al modelo y utilizando cualquier función suya, nos ahorramos el hecho de copiar y pegar la función en los distintos archivos (en caso de seguir trabajando sin la arquitectura MVC).

En cuanto al mantenimiento, al estar separado todo en distintos archivos, si tú quieres modificar una función solo la tienes que modificar en ese archivo ya que va a ser reutilizado o si modificas el HTML, solo modificas el HTML sin tener que, a lo mejor, modificar también parte de la lógica al estar todo junto.

Por último, con las pruebas aquí se gana mucho tiempo, ya que nos podemos crear un archivo en el que instanciamos el objeto y la función que queramos probar para ver si funciona bien y un pequeño HTML para comprobar esa función, en vez de ejecutar todo el archivo con todas las funciones y hacer todo el recorrido hasta llegar a la que queremos comprobar.